

MINI ASSESSMENT

API ERROR HANDLING AND SAFE JSON ACCESS

Name	Nallapureddy MohanReddy
Reg. No.	99230040690
Email	99230040690@klu.ac.in
Department	CSE – AIML

1. If an API returns 429, what should your program do?

API Response
Status = 429
Read Retry-After
Wait / Backoff
Retry safely

Answer

A 429 response means the server is rate-limiting the client. The program should pause instead of immediately sending another request. If Retry-After is supplied, follow it; otherwise, use exponential backoff. After a reasonable retry limit, stop and handle the failure gracefully.

2. If response.json() fails, what should your program do?

Response received
Attempt JSON parsing
Parsing fails
Catch exception
Inspect status / text and handle safely

Answer

The program should catch the JSON parsing exception so an unexpected response does not crash the application. It should check the HTTP status and, when useful, inspect the response text for debugging. The operation should then recover safely or terminate in a controlled manner.

3. If a JSON key is missing, why might `.get()` be safer than `[]`?

JSON object
Key may be absent
Use <code>data.get("status", "unknown")</code>
Return value / default
Avoid <code>KeyError</code>

Answer

`data.get("status")` is safer when a field is optional because it returns `None` when the key is absent, or a supplied default when specified. In contrast, `data["status"]` raises `KeyError` when the field does not exist. This makes `.get()` useful for uncertain API responses.

QUICK COMPARISON

Method	Key exists	Key missing
<code>data.get("status", "unknown")</code>	Returns value	Returns "unknown"
<code>data["status"]</code>	Returns value	Raises <code>KeyError</code>

KEY TAKEAWAY

Reliable API programs should anticipate rate limits, invalid response formats, and missing JSON fields. Explicit error handling and safe access patterns help prevent unexpected failures.